

Rapporto di Progetto – MeteoAPP

Sommario

Rapporto di Progetto – MeteoAPP	<i>Errore. Il segnalibro non è definito.</i>
Introduzione.....	1
Obiettivi del progetto	2
Tecnologie autilizate	3
Frontend	3
Backend Mobile	5
Backend Server	7
Tunelling e sviluppo locale	10
Metdologie	11
Pattern MVVM.....	11
API RESTful.....	11
Git.....	11
Postman e Swagger.....	11
Ngrok	12
Firebase Admin SDK.....	12
Sintesi del processo di implementazione e delle principali sfide incontrate.....	12
Fase 1 – Struttura di base e persistenza locale	12
Fase 2 – Sincronizzazione cloud e configurazioni utente.....	12
Fase 3 – Backend API e invio notifiche automatiche	13
Testing su dispositivi fisici e utilizzo di Ngrok.....	13
Funzionalità aggiuntive e sfide affrontate	13
Schermate dell'Applicazione	14
Schermata iniziale e menu laterale.....	14
Lista delle città salvate e ricerca dinamica.....	14
Configurazione delle soglie di temperatura.....	15
Aggiunta località da mappa interattiva	15
Dettaglio meteo di una località selezionata.....	16
Scheda meteo semplificata	16
Grafico giornaliero delle temperature	17
Visualizzazione circolare del vento.....	17
Probabilità di pioggia.....	18
Visualizzazione pressione atmosferica	18
Breve valutazione del risultato finale e dei possibili miglioramenti	19

Introduzione

MeteoAPP è un applicazione mobile cross-platform sviluppata utilizzando .NET MAUI. L'obiettivo del progetto è offrire agli utenti un sistema semplice ma avanzato per ottenere dati meteorologici aggiornati, configurare soglie personalizzate di notifiche e gestire località tramite mappa e GPS.

L'app è pensata per essere reattiva, affidabile, personalizzabile e cloud-aware, grazie il sistema di Appwrite per la sincronizzazione e Firebase per la gestione delle notifiche.



Obiettivi del progetto

Gli obiettivi principali perseguiti nel progetto sono stati molteplici e hanno riguardato diversi aspetti dell'interazione utente con i dati meteorologici, con l'intento di offrire un'esperienza completa, personalizzata e tecnologicamente avanzata. In primo luogo, è stata implementata la visualizzazione in tempo reale delle condizioni meteo attraverso l'utilizzo dell'API di OpenWeather, consentendo all'utente di accedere rapidamente a informazioni aggiornate su temperatura, umidità, vento e altri parametri atmosferici. Parallelamente, si è lavorato sulla gestione di una lista personalizzata di città salvate dall'utente, i cui dati vengono memorizzati localmente tramite un database SQLite per garantire accessibilità anche offline. È stata prevista inoltre la possibilità di aggiungere manualmente nuove località sia attraverso una funzione di ricerca testuale, sia tramite la selezione diretta su una mappa interattiva, ampliando così la flessibilità dell'applicazione. Il rilevamento automatico della posizione geografica mediante GPS rappresenta un'altra funzionalità chiave, che permette di fornire dati contestualizzati in base alla posizione attuale dell'utente. A livello di architettura cloud, si è integrato Appwrite per la sincronizzazione remota e sicura dei dati utente, così da consentire un accesso coerente da più dispositivi. È stata inoltre data la possibilità di configurare soglie di temperatura personalizzate, in modo da ricevere notifiche push tempestive e mirate tramite Firebase, garantendo un alto grado di reattività e personalizzazione. Infine, è stato previsto un sistema di monitoraggio periodico lato server delle condizioni meteo relative alle località salvate, al fine di assicurare un aggiornamento costante e automatizzato delle informazioni anche senza l'interazione diretta dell'utente.

Tecnologie utilizzate

Frontend

.NET MAUI

.NET MAUI è stato scelto come framework principale per lo sviluppo dell'interfaccia utente nativa multiplatforma. Grazie alla sua struttura unificata, è stato possibile scrivere un'unica codebase in C# per Android, iOS e Desktop, mantenendo al contempo performance elevate e pieno accesso alle funzionalità native dei dispositivi. Nel cuore dell'applicazione troviamo la classe App, che funge da entry point, dove vengono istanziati i servizi principali come il database locale (**DatabaseService**) e il sistema di sincronizzazione cloud tramite Appwrite (**AppwriteSyncService**).

```
public App()
{
    InitializeComponent();
    DatabaseService = new DatabaseService();
    AppwriteSyncService = new AppwriteSyncService(DatabaseService);
}
```

L'inizializzazione avviene nel metodo OnStart, assicurando che i dati locali e remoti siano coerenti già all'avvio.

```
protected override async void OnStart()
{
    if (DatabaseService != null)
    {
        try
        {
            await DatabaseService.InitializeAsync();
            await AppwriteSyncService!.PullCitiesFromAppwriteAsync();
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine($"Errore inizializzazione App: {ex.Message}");
        }
    }
}
```

Blazor WebView

L'integrazione di componenti web all'interno dell'app è stata possibile grazie al controllo BlazorWebView, che consente di ospitare interfacce Web realizzate in Blazor direttamente in una pagina .NET MAUI.

```
<blazor:BlazorWebView HostPage="wwwroot/index.html">
    <blazor:BlazorWebView.RootComponents>
        <blazor:RootComponent Selector="#app" ComponentType="{x:Type local:WeatherInfo}" />
    </blazor:BlazorWebView.RootComponents>
</blazor:BlazorWebView>
```

Chart.js

Per la visualizzazione grafica delle condizioni meteo è stata integrata la libreria Chart.js, sfruttata per rappresentare sia l'andamento della temperatura giornaliera (grafico a linee), sia altri dati atmosferici come vento, pioggia e pressione (grafici a ciambella).

L'integrazione è stata effettuata tramite Blazor WebView, consentendo di combinare componenti Web dinamici all'interno di un'app .NET MAUI nativa.

Tra i grafici implementati, uno dei più significativi è quello delle temperature nelle diverse fasce orarie (mattina, pomeriggio, sera, notte).

```
window.renderTempChart = (temps) => {
    new Chart(document.getElementById("tempChart").getContext("2d"), {
        type: "line",
        data: {
            labels: ["Morning", "Afternoon", "Evening", "Night"],
            datasets: [{
                label: "Temp (°C)",
                data: temps,
                borderColor: "teal",
                backgroundColor: "rgba(0, 128, 128, 0.2)",
                fill: true,
                tension: 0.4
            }]
        },
        options: { plugins: { legend: { display: false } } }
    });
};
```

Leaflet.js

Per la visualizzazione grafica delle condizioni meteo è stata integrata la libreria **Chart.js**, sfruttata per rappresentare sia l'andamento della temperatura giornaliera (grafico a linee), sia altri dati atmosferici come vento, pioggia e pressione (grafici a ciambella).

L'integrazione è stata effettuata tramite Blazor WebView, consentendo di combinare componenti Web dinamici all'interno di un'app .NET MAUI nativa. Tra i grafici implementati, uno dei più significativi è quello delle temperature nelle diverse fasce orarie (mattina, pomeriggio, sera, notte).

```

let map;
let marker;
let lat = 46.0334565;
let lon = 8.9675471;

function initMap() {
    map = L.map('map', {
        center: [lat, lon],
        zoom: 13,
        zoomControl: true,
        attributionControl: true,
        touchZoom: true,
        doubleClickZoom: true,
        scrollWheelZoom: false
    });

    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        attribution:
            '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors'
    }).addTo(map);

    function sendCoordsToHost(lat, lon) {
        var data = JSON.stringify({ lat: lat, lng: lon });
        window.location.href = "js://update-location/" + encodeURIComponent(data);
    }
}

```

Backend Mobile

SQLite

Per garantire la persistenza dei dati in locale, anche in assenza di connettività, è stato integrato SQLite, una soluzione leggera ma estremamente efficace per il salvataggio strutturato delle informazioni utente. In particolare, l'app salva localmente l'elenco delle località preferite selezionate o aggiunte dall'utente, evitando così ogni volta il caricamento da remoto. La gestione del database avviene all'interno della classe DatabaseService, che incapsula tutte le operazioni relative alla creazione, inizializzazione, inserimento, aggiornamento e lettura dei dati. Un aspetto fondamentale è la gestione del percorso del file .db3, che viene posizionato nella directory locale dell'app, garantendo compatibilità e sicurezza tra le diverse piattaforme (Android, iOS, desktop). Il costruttore della classe si occupa proprio di questa inizializzazione di base, creando la connessione asincrona al database e assicurandosi che il percorso di salvataggio esista, creando eventualmente la directory se mancante. Questo approccio rende il sistema robusto già dalla fase di avvio e previene potenziali errori legati al file system.

```

public DatabaseService()
{
    try
    {
        var dbPath = Path.Combine(
            Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData),
            "meteo.db3"
        );

        var directoryPath = Path.GetDirectoryName(dbPath);
        if (!Directory.Exists(directoryPath))
        {
            try
            {
                Directory.CreateDirectory(directoryPath!);
            }
            catch (Exception dirEx)
            {
                Debug.WriteLine($"Directory creation error: {dirEx.Message}");
            }
        }

        _databaseConnection = new SQLiteAsyncConnection(dbPath);
    }
    catch (Exception ex)
    {
        Debug.WriteLine($"Initial Error DatabaseService: {ex.Message}");
        throw;
    }
}

```

Appwrite SDK

L'integrazione con Appwrite ha rappresentato una componente chiave per abilitare la sincronizzazione in cloud dei dati utente, in particolare per salvare e recuperare le città preferite e le relative impostazioni da diversi dispositivi. Questo ha permesso di mantenere coerenza tra i dati locali e remoti, offrendo un'esperienza utente fluida e personalizzata. Il servizio AppwriteSyncService incapsula tutta la logica di interazione con Appwrite e viene inizializzato centralmente all'avvio dell'app. Una delle parti fondamentali è proprio il costruttore della classe, in cui vengono caricati i dati di configurazione dal file config.json e si stabilisce la connessione con il progetto Appwrite tramite Client e Databases. In questo punto si impostano anche i riferimenti a databaseId e collectionId, che rappresentano gli elementi chiave per accedere ai documenti utente. Questo approccio modulare consente all'app di spingere e ricevere dati dal cloud in modo sicuro, asincrono e multi-device.

```

public AppwriteSyncService(DatabaseService databaseService)
{
    _databaseService = databaseService;

    var config = LoadConfig();

    _client = new Client()
        .SetEndpoint("https://cloud.appwrite.io/v1")
        .SetProject(config.AppwriteProjectId)
        .SetKey(config.AppwriteApiKey);

    _databases = new Databases(_client);
    _databaseId = config.AppwriteDatabaseId;
    _collectionId = config.AppwriteCollectionId;
}

```

Firebase Cloud Messaging + Plugin.Firebase.CloudMessaging

Per la gestione delle notifiche push personalizzate, l'app fa uso di Firebase Cloud Messaging (FCM), un servizio altamente scalabile che consente l'invio di messaggi mirati verso dispositivi mobili. L'integrazione è stata resa possibile grazie al plugin *Plugin.Firebase.CloudMessaging*, specifico per .NET MAUI, che offre un'interfaccia semplice per ottenere il token FCM del dispositivo, registrarsi per le notifiche e gestire le preferenze localmente. Questa architettura consente all'utente di configurare soglie personalizzate di temperatura per ciascuna località e ricevere notifiche automatiche dal backend ogni volta che tali soglie vengono superate. La ViewModel *NotificationSettingsViewModel* gestisce tutta la logica relativa alla configurazione e sincronizzazione delle preferenze, comprese le chiamate al server e la persistenza delle impostazioni.

Il componente centrale di questo meccanismo è il token univoco generato da Firebase, che identifica il dispositivo e viene associato alle preferenze utente. Il token viene recuperato dinamicamente, salvato nelle preferenze e poi utilizzato per la registrazione delle soglie sul backend.

```

string token = string.Empty;
try
{
    await CrossFirebaseCloudMessaging.Current.CheckIfValidAsync();
    token = await CrossFirebaseCloudMessaging.Current.GetTokenAsync();
    Console.WriteLine("Token Firebase ottenuto: " + token);
}
catch (Exception ex)
{
    Console.WriteLine("Errore nell'ottenere il token Firebase: " + ex.Message);
}

```

Backend Server

ASP.NET Core Web API

Per consentire la comunicazione tra l'app mobile e il backend cloud, è stata sviluppata una Web API in ASP.NET Core, progettata per ricevere, salvare e aggiornare le impostazioni personalizzate relative alle notifiche meteo configurate dagli utenti. Il controller principale, *NotificationSettingsController*, espone due endpoint: uno GET per il recupero delle preferenze in base a token e località, e uno POST per la loro creazione o aggiornamento. Il backend utilizza Firestore (tramite Firebase Admin SDK) come database NoSQL per

archiviare in modo scalabile le preferenze, garantendo letture e scritture efficienti anche su larga scala. Questo approccio permette al sistema di mantenere sincronizzate le soglie impostate dall'utente e attivare notifiche personalizzate in base ai valori di temperatura.

Firestore

Le impostazioni personalizzate degli utenti, come le soglie di temperatura e le preferenze per le notifiche, vengono archiviate in **Firestore**, il database NoSQL scalabile di Firebase. Grazie alla sua natura document-based e alla facile integrazione con l'ecosistema .NET tramite il pacchetto Google.Cloud.Firestore, è stato possibile strutturare i dati in modo efficiente, mantenendo performance elevate anche in presenza di numerosi dispositivi e utenti. L'API backend scritta in ASP.NET Core interagisce direttamente con Firestore, salvando e aggiornando i dati in base alla combinazione Token + Location, così da assicurare che ogni utente riceva notifiche personalizzate solo per le località a cui è interessato.

```
[ApiController]
[Route("api/[controller]")]
1 reference
public class NotificationSettingsController : ControllerBase
{
    3 references
    private readonly FirestoreDb _firestoreDb;
    0 references
    public NotificationSettingsController(FirestoreDb firestoreDb)
    {
        _firestoreDb = firestoreDb;
    }
    [HttpPost]
    0 references
    public async Task<IActionResult> Post([FromBody] UserNotificationSettings[] settings)
    {
        if (settings == null || settings.Length == 0)
            return BadRequest("Payload non valido.");
        var settingsRef = _firestoreDb.Collection("settings");
        foreach (var setting in settings)
        {
            var query = settingsRef
                .WhereEqualTo("Token", setting.Token)
                .WhereEqualTo("Location", setting.Location);
            var snapshot = await query.GetSnapshotAsync();
            if (snapshot.Count > 0)
            {
                foreach (var doc in snapshot.Documents)
                {
                    var docRef = settingsRef.Document(doc.Id);
                    var updates = new Dictionary<string, object>
                    {
                        { "TemperatureMax", setting.TemperatureMax },
                        { "TemperatureMin", setting.TemperatureMin },
                        { "IsEnabled", setting.IsEnabled }
                    };
                    await docRef.UpdateAsync(updates);
                }
            }
            else
            {
                var newDoc = settingsRef.Document();
                await newDoc.SetAsync(setting);
            }
        }
        return Ok(new { message = "Impostazioni salvate o aggiornate con successo." });
    }
}
```


Firebase Admin SDK

Sul backend, è stato integrato Firebase Admin SDK per abilitare l'invio automatico delle notifiche push, basandosi sui dati personalizzati salvati dagli utenti nel database Firestore. Questo SDK consente una comunicazione sicura e autenticata con i servizi Firebase utilizzando una chiave di servizio (serviceAccountKey.json). L'applicazione backend ASP.NET Core inizializza Firebase all'avvio, configura le credenziali e crea un'istanza condivisa di FirestoreDb, che viene poi iniettata tramite dependency injection. Inoltre, viene avviato un servizio HostedService dedicato al monitoraggio delle soglie di temperatura, il quale sfrutta queste credenziali per leggere le impostazioni salvate e notificare i dispositivi registrati. Questo approccio rende il backend autonomo nell'interazione con Firebase, senza necessità di intervento diretto da parte dell'utente finale.

```
var keyPath = Path.Combine(AppContext.BaseDirectory, "serviceAccountKey.json");
var credential = GoogleCredential.FromFile(keyPath);
FirebaseApp.Create(new AppOptions
{
    Credential = credential
});
string json = File.ReadAllText(keyPath);
using var doc = JsonDocument.Parse(json);
string? projectId = doc.RootElement.GetProperty("project_id").GetString();
```

HostedService in C#

Nel backend ASP.NET Core è stato implementato un servizio in background (HostedService) per monitorare periodicamente le condizioni meteo delle località salvate dagli utenti. Questo servizio, TemperatureMonitoringService, si attiva all'avvio dell'applicazione e opera in modo ciclico ogni 5 minuti. Accede al database Firestore per leggere le impostazioni salvate (come soglie di temperatura massima e minima) e confrontarle con i dati meteo attuali. Se viene rilevato uno sfioramento delle soglie configurate, viene inviata una notifica push al dispositivo dell'utente tramite Firebase Cloud Messaging. In questo modo, l'intero processo di controllo e notifica avviene in modo completamente automatico e continuo, senza bisogno di interazioni manuali da parte dell'utente.

```

protected override async Task ExecuteAsync(Cancellation_token stoppingToken)
{
    TimeSpan interval = TimeSpan.FromMinutes(5);

    while (!stoppingToken.IsCancellationRequested)
    {
        try
        {
            var settingsRef = _firestoreDb.Collection("settings");
            var query = settingsRef.WhereEqualTo("IsEnabled", true);
            var snapshot = await query.GetSnapshotAsync();

            foreach (var doc in snapshot.Documents)
            {
                var settings = doc.ConvertTo<UserNotificationSettings>();

                double currentTemp = await GetTemperatureForLocationAsync(settings.Location);

                if (currentTemp > settings.TemperatureMax || currentTemp < settings.TemperatureMin)
                {
                    await SendTemperatureAlertAsync(settings.Token, settings.Location,
                        currentTemp, settings.TemperatureMax, settings.TemperatureMin);
                }
            }
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Errore durante il monitoraggio della temperatura.");
        }

        await Task.Delay(interval, stoppingToken);
    }
}

```

Tunelling e sviluppo locale

Ngrok

Durante la fase di sviluppo e test dell'applicazione mobile su dispositivi fisici, si è reso necessario esporre il backend locale sviluppato in ASP.NET Core verso l'esterno. Questo perché i dispositivi Android o iOS, pur collegati alla stessa rete, non possono accedere direttamente a localhost della macchina di sviluppo. Per risolvere questo problema è stato utilizzato Ngrok, uno strumento che consente di creare un tunnel HTTPS sicuro verso una porta locale.

Attraverso un semplice comando terminale, Ngrok mappa la porta su cui gira il backend (dotnet run) e fornisce un URL pubblico temporaneo, accessibile da qualsiasi dispositivo connesso a Internet. Questo ha permesso all'app mobile di effettuare richieste HTTP/HTTPS al server locale come se fosse già in produzione.

Esempio di utilizzo:

cd /percorso/backend

dotnet run

ngrok http 5000

Output generato da Ngrok:

Forwarding https://ec0d-2a02-1210-6029-2600.ngrok-free.app -> http://localhost:5000

L'URL pubblico generato (https://...ngrok-free.app) è stato poi inserito direttamente nel codice dell'app .NET MAUI, in particolare all'interno della classe NotificationSettingsViewModel, per definire il punto di accesso alle API:

`private readonly string baseUrl = "https://ec0d-2a02-1210-6029-2600.ngrok-free.app/api/NotificationSettings";`

Questo approccio ha facilitato lo sviluppo, il debugging e il testing delle funzionalità cloud, come il salvataggio delle impostazioni utente o la ricezione delle soglie meteo, direttamente da dispositivi reali.

Ogni volta che Ngrok viene riavviato, viene generato un nuovo URL casuale, a meno che non si utilizzi la versione PRO, che consente di configurare un sottodominio statico. Per evitare modifiche frequenti al codice o alla configurazione, è consigliato utilizzare la versione PRO o gestire dinamicamente il base URL tramite configurazioni esterne.

Metdologie

Pattern MVVM

Nel frontend sviluppato con .NET MAUI è stato adottato il pattern MVVM (Model-View-ViewModel), uno dei più comuni e raccomandati per lo sviluppo di applicazioni moderne multiplatforma. Questo approccio consente una netta separazione tra la logica dell'interfaccia utente (View), la logica applicativa (ViewModel) e i dati (Model). Grazie a MVVM, è possibile mantenere un codice più ordinato, testabile e riutilizzabile. Le ViewModel si occupano del binding dei dati e della gestione degli eventi, mentre la View rimane semplice e responsabile solo della presentazione. Questo ha facilitato l'integrazione di funzionalità come la modifica delle soglie di temperatura o l'attivazione/disattivazione delle notifiche, mantenendo un codice facilmente manutenibile.

API RESTful

Per la comunicazione tra l'app mobile e il backend, è stata progettata una Web API RESTful utilizzando ASP.NET Core. Questo tipo di architettura segue il paradigma delle operazioni HTTP standard (GET, POST, PUT, DELETE), consentendo una comunicazione semplice, scalabile e compatibile con qualsiasi client. Ogni endpoint rappresenta una risorsa logica e restituisce dati in formato JSON, garantendo una chiara separazione tra le operazioni e una maggiore interoperabilità. La struttura REST ha permesso di gestire facilmente operazioni come il salvataggio, l'aggiornamento e il recupero delle preferenze utente legate alle notifiche meteo.

Git

Durante lo sviluppo del progetto, è stato utilizzato Git come sistema di controllo di versione distribuito. Grazie a Git, ogni modifica è stata tracciata in modo sicuro e reversibile, permettendo di collaborare in modo efficace, creare rami per funzionalità specifiche e gestire facilmente l'evoluzione del codice nel tempo. L'adozione di Git ha anche facilitato il debugging, il versionamento delle release e l'integrazione continua di nuove funzionalità senza compromettere la stabilità dell'applicazione.

Postman e Swagger

Per testare, documentare e verificare il corretto funzionamento delle API RESTful, sono stati utilizzati due strumenti fondamentali: Postman e Swagger. Postman ha permesso di simulare richieste HTTP verso gli endpoint dell'API, verificando così in modo rapido e interattivo che le risposte fossero corrette. Inoltre, il backend è stato configurato per generare automaticamente una documentazione tramite Swagger (OpenAPI), accessibile via browser. Swagger consente di esplorare gli endpoint, leggere le specifiche dei parametri richiesti e testare le API in tempo reale, migliorando la comprensione e la validazione del sistema anche da parte di terzi.

Ngrok

Durante lo sviluppo e il test su dispositivi fisici, è stato utilizzato Ngrok, uno strumento essenziale per esporre le API locali (hostate su localhost) tramite un URL pubblico accessibile da qualsiasi rete. Questo ha consentito all'app mobile, in esecuzione su dispositivi Android e iOS, di comunicare con il backend ASP.NET Core in esecuzione sul computer di sviluppo. Ngrok ha rappresentato una soluzione temporanea ma efficace per testare in condizioni reali le chiamate HTTP, la ricezione dei dati e la configurazione delle notifiche. Ogni sessione generava un URL univoco, aggiornato nel codice durante i test, a meno di utilizzare la versione PRO per un dominio fisso.

Firestore Admin SDK

Infine, il backend sfrutta il Firestore Admin SDK per l'invio automatico delle notifiche push. Questo SDK, integrato nel backend ASP.NET Core, consente di inviare messaggi FCM (Firebase Cloud Messaging) in modo sicuro e autenticato, utilizzando un file di servizio (serviceAccountKey.json). Il sistema legge le impostazioni salvate nel database Firestore e, tramite un servizio in background, verifica ciclicamente se le soglie di temperatura sono state superate. In caso positivo, il backend invia notifiche direttamente al dispositivo dell'utente, in base al token salvato, garantendo un'esperienza proattiva e completamente automatizzata.

Sintesi del processo di implementazione e delle principali sfide incontrate

Lo sviluppo dell'applicazione ha seguito un processo iterativo, suddiviso in fasi ben distinte e guidato da un approccio modulare. Questa struttura ha permesso di introdurre progressivamente le funzionalità, mantenendo al tempo stesso chiarezza architetturale e facilità di manutenzione. Il progetto è stato articolato in tre principali blocchi: frontend mobile, backend server e sincronizzazione cloud, ciascuno progettato per integrarsi fluidamente con gli altri.

Fase 1 – Struttura di base e persistenza locale

La prima fase ha previsto la realizzazione della struttura principale dell'applicazione attraverso il framework .NET MAUI, con l'adozione del pattern MVVM per separare efficacemente la logica di presentazione, quella di business e la gestione dei dati. In questa fase è stato anche introdotto SQLite per il salvataggio persistente delle località preferite. La creazione del database e delle relative tabelle è stata gestita in modo asincrono e multiplatforma, con particolare attenzione al percorso dei file e alla corretta inizializzazione in fase di avvio. Ciò ha garantito che l'app fosse pienamente operativa anche in modalità offline.

Fase 2 – Sincronizzazione cloud e configurazioni utente

Una volta definita la struttura base dell'applicazione, è stata integrata la sincronizzazione cloud tramite Appwrite, con una configurazione dinamica dei parametri di connessione (project ID, database ID, collection ID) gestita attraverso il file config.json. In Appwrite, ogni città salvata viene rappresentata da un documento contenente i seguenti campi:

- idCity (Integer)
- name (String)
- country (String)
- latitude e longitude (Double)
- device (String, identificativo del dispositivo)

La gestione di lettura e scrittura dei dati è affidata al servizio AppwriteSyncService, che verifica la presenza di documenti esistenti per una data combinazione idCity + device ed evita duplicazioni, creando un nuovo documento solo se necessario. Questo approccio ha garantito una sincronizzazione precisa e personalizzata per ogni dispositivo. In parallelo, è stata sviluppata una funzionalità per la configurazione delle notifiche personalizzate: l'utente può definire soglie di temperatura massima e minima per ciascuna località e ricevere notifiche automatiche in caso di superamento. Per supportare questa logica, è stata integrata Firebase Cloud Messaging nel frontend, con gestione dinamica dei token del dispositivo e salvataggio delle preferenze tramite API RESTful dedicate.

Fase 3 – Backend API e invio notifiche automatiche

Il backend ASP.NET Core espone un set di API RESTful progettate per ricevere, salvare e aggiornare le preferenze utente relative alle notifiche meteo. Queste impostazioni vengono archiviate nel database Firebase Firestore, sfruttando il Firebase Admin SDK per l'accesso autenticato tramite chiave di servizio.

Ogni documento della collezione Firestore settings contiene le seguenti proprietà:

- Location: ad esempio "Milano, Italia"
- TemperatureMax e TemperatureMin: soglie personalizzate
- IsEnabled: attivazione o disattivazione delle notifiche
- Token: identificativo univoco del dispositivo generato da Firebase Cloud Messaging (FCM)

L'inserimento o aggiornamento di questi dati avviene tramite l'endpoint POST del controller NotificationSettingsController, che verifica la presenza di un documento già esistente sulla base della coppia Token + Location. Se il documento esiste, viene aggiornato; in caso contrario, ne viene creato uno nuovo.

Una delle componenti chiave del sistema è il servizio in background (HostedService), che si occupa del monitoraggio automatico delle condizioni meteo. Ogni cinque minuti, il servizio interroga Firestore per recuperare tutte le configurazioni attive (IsEnabled = true) e confronta la temperatura attuale (reale o simulata) con le soglie impostate dall'utente. Se una delle condizioni configurate viene superata, il sistema genera e invia automaticamente una notifica push al dispositivo, sfruttando Firebase Cloud Messaging.

Testing su dispositivi fisici e utilizzo di Ngrok

Durante i test su dispositivi reali Android e iOS, è emersa la necessità di esporre il backend locale verso l'esterno. A tale scopo è stato utilizzato Ngrok, che ha consentito di creare un tunnel HTTPS pubblico verso il server locale, permettendo ai dispositivi di effettuare richieste API in tempo reale. Una sfida rilevante in questa fase è stata la gestione dinamica dell'URL Ngrok, che cambia ad ogni sessione gratuita, obbligando a modificare manualmente il codice dell'app o a introdurre un sistema di configurazione esterno.

Funzionalità aggiuntive e sfide affrontate

Una delle funzionalità aggiunte durante il progetto è stata la barra di ricerca delle località, non prevista inizialmente, ma rivelatasi cruciale per migliorare l'usabilità. L'integrazione della SearchBar nella pagina ListMeteoPage ha introdotto una sfida legata alle prestazioni durante la digitazione e all'aggiornamento in tempo reale dei risultati.

```
<SearchBar
  x:Name="CitySearchBar"
  Placeholder="Search for cities..."
  Text="{Binding SearchText, UpdateSourceEventName=TextChanged}"
  VerticalOptions="Start"
  Margin="10,10,10,0"
  Grid.Row="0" />
```

Con binding alla proprietà SearchText nella ViewModel e filtraggio automatico sulla collezione FilteredCities. È stato necessario ottimizzare attentamente l'aggiornamento delle liste per evitare lag, flickering o duplicazioni visive nella ListView. Questa ottimizzazione ha incluso debounce logico, filtraggio case-insensitive e gestione asincrona del caricamento.

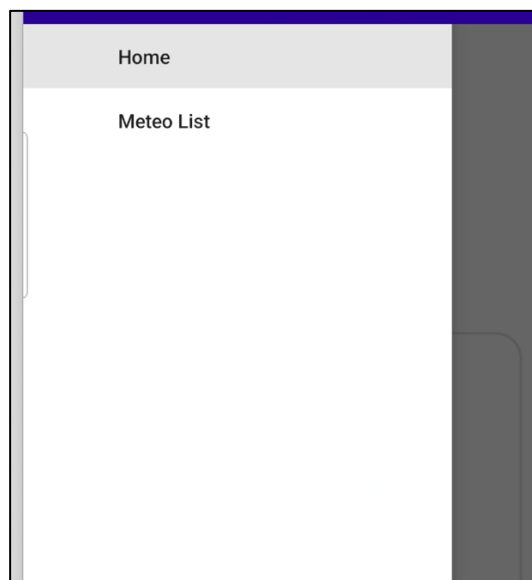
Un'altra sfida significativa ha riguardato la gestione dei token FCM: è stato fondamentale assicurarsi che il token venisse generato correttamente, recuperato anche in caso di errori temporanei, e associato correttamente alle impostazioni salvate nel cloud.

Schermate dell'Applicazione

L'interfaccia di MeteoAPP è stata progettata per essere semplice, funzionale e visivamente pulita. Di seguito vengono presentate alcune schermate rappresentative che illustrano le principali funzionalità dell'applicazione.

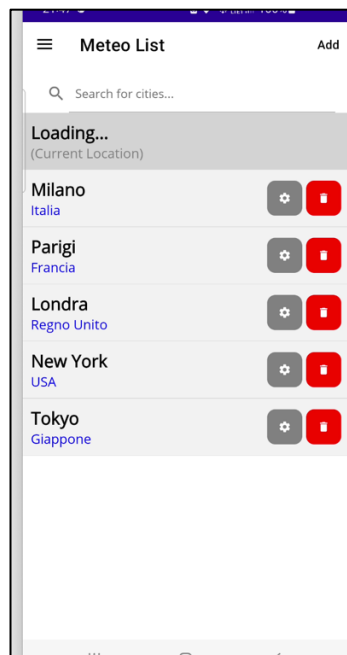
Schermata iniziale e menu laterale

All'apertura dell'app, l'utente ha accesso a un menu laterale (hamburger menu) che permette di navigare tra le sezioni principali: Home e Lista delle Città. Questo approccio offre una navigazione chiara e coerente, con accesso rapido alle funzionalità più utilizzate.



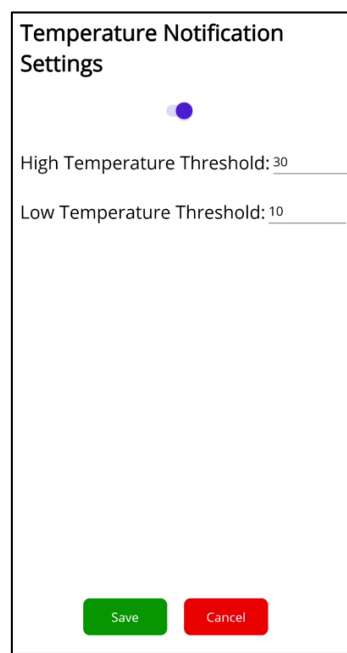
Lista delle città salvate e ricerca dinamica

La schermata "Meteo List" mostra tutte le città salvate dall'utente, ognuna con possibilità di modifica o eliminazione. In alto è integrata una barra di ricerca, legata alla ViewModel, che aggiorna in tempo reale i risultati filtrando la lista. La performance della ricerca è stata ottimizzata per grandi dataset e input rapidi.



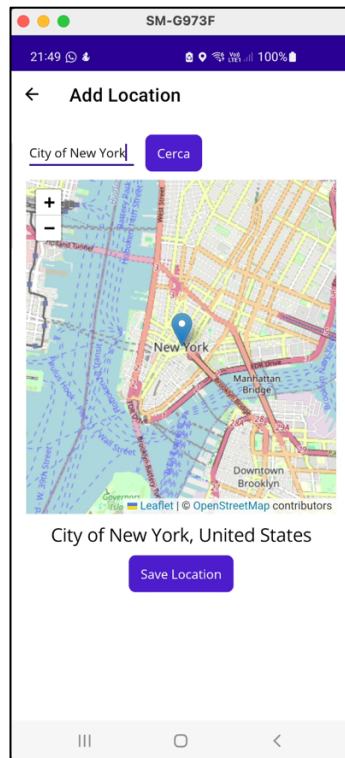
Configurazione delle soglie di temperatura

Ogni città può essere configurata con soglie personalizzate di temperatura, impostando i valori minimo e massimo oltre i quali ricevere una notifica. La schermata include uno switch per abilitare/disabilitare le notifiche. Le preferenze vengono salvate in locale e sincronizzate sul backend.



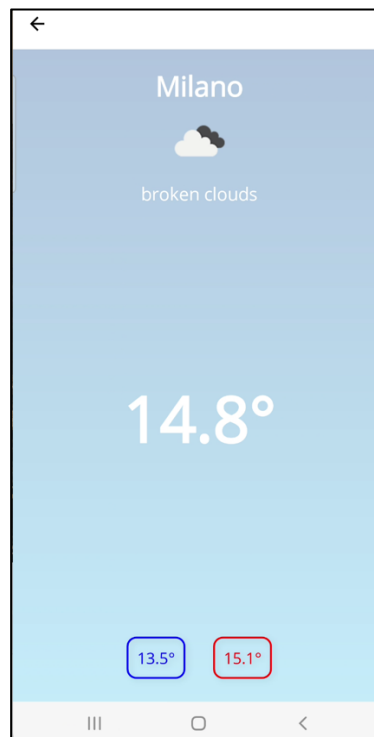
Aggiunta località da mappa interattiva

Per aggiungere nuove località, è disponibile una schermata con **mappa interattiva** basata su Leaflet.js. Dopo aver digitato il nome della città, l'utente può selezionare visivamente la posizione e salvarla. Questa funzionalità arricchisce l'app con un approccio visivo e intuitivo.



Dettaglio meteo di una località selezionata

Selezionando una città, l'utente accede a una schermata dettagliata che mostra le condizioni meteo correnti: temperatura, icona meteo e soglie impostate. Il design è minimale ma informativo, con colori che evidenziano le soglie superate.



Scheda meteo semplificata

Questa schermata rappresenta una scheda meteo riassuntiva che mostra le informazioni essenziali in formato compatto: temperatura, descrizione e valori min/max. È pensata per una visualizzazione rapida e immediata.

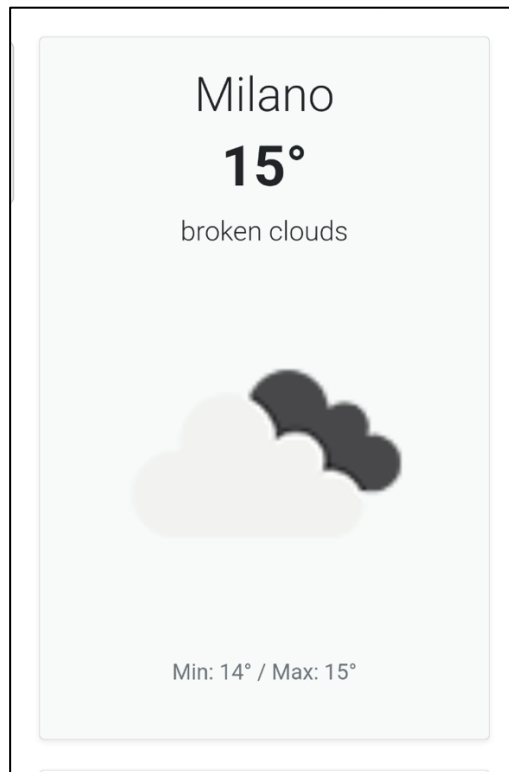
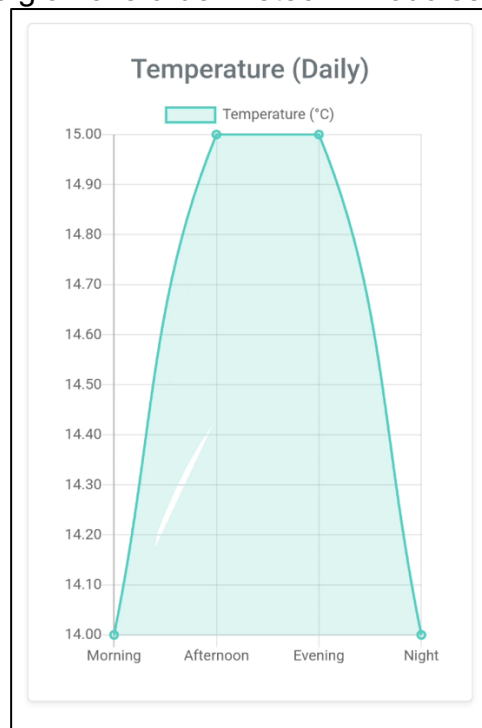


Grafico giornaliero delle temperature

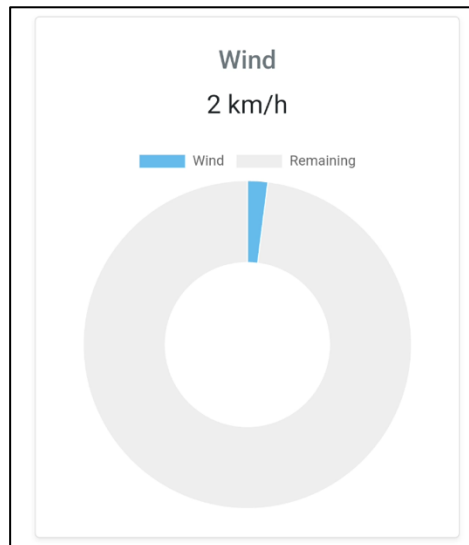
Il grafico a linee generato con Chart.js rappresenta l'andamento della temperatura durante le fasce orarie (mattina, pomeriggio, sera, notte). Questo tipo di visualizzazione consente di comprendere l'evoluzione giornaliera del meteo in modo semplice e visivo.



Visualizzazione circolare del vento

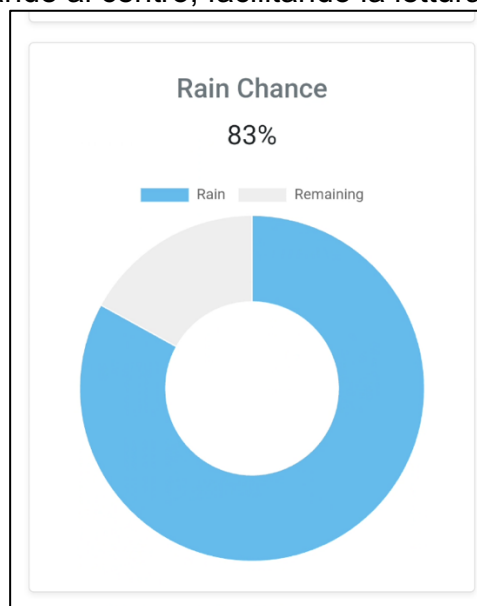
Un altro grafico a ciambella mostra la velocità del vento attuale. La parte blu rappresenta l'intensità rilevata, mentre il resto indica il range restante. Questo tipo di visualizzazione

consente di percepire visivamente la proporzione del valore attuale rispetto al massimo atteso.



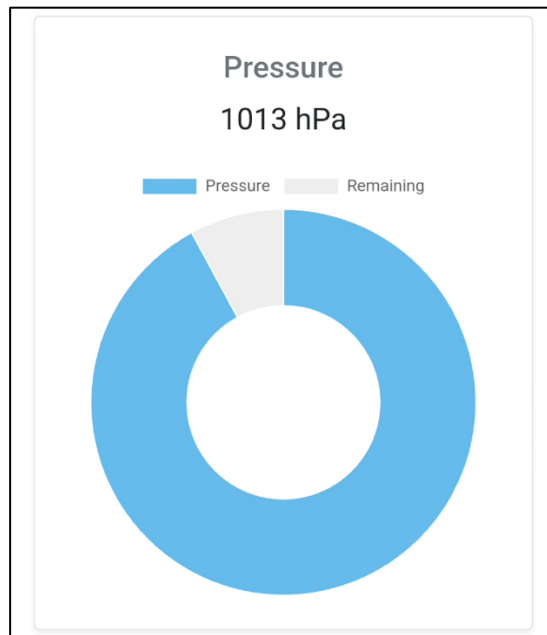
Probabilità di pioggia

Sempre con un grafico a ciambella, viene mostrata la percentuale di possibilità di pioggia. Il valore è evidenziato in grande al centro, facilitando la lettura anche su dispositivi mobili.



Visualizzazione pressione atmosferica

Infine, la pressione atmosferica è rappresentata anch'essa tramite un grafico circolare. Questo contribuisce a fornire una panoramica completa delle condizioni atmosferiche, integrando estetica e chiarezza.



Breve valutazione del risultato finale e dei possibili miglioramenti

L'applicazione MeteoAPP ha raggiunto pienamente gli obiettivi iniziali, offrendo una piattaforma mobile intuitiva, reattiva e ben integrata con servizi cloud moderni. Le funzionalità principali come la visualizzazione meteo in tempo reale, la gestione delle località preferite, la configurazione delle soglie di notifica e la sincronizzazione tra dispositivi sono state implementate con successo, garantendo un'esperienza utente solida e coerente. L'interfaccia, sviluppata con .NET MAUI e ottimizzata con componenti dinamici come Chart.js e Leaflet, ha permesso di offrire visualizzazioni grafiche coinvolgenti e interazioni fluide, anche su dispositivi meno performanti. L'integrazione con Firebase e Appwrite ha assicurato funzionalità cloud avanzate, mantenendo al tempo stesso semplicità nel deployment e nella scalabilità.

Tuttavia, alcuni aspetti potrebbero essere ulteriormente migliorati nelle versioni future. Ad esempio, l'attuale gestione delle notifiche push potrebbe essere estesa per supportare più condizioni meteo (es. pioggia, vento forte), offrendo una maggiore personalizzazione. Anche l'aggiornamento automatico dei dati in background sul dispositivo, senza necessità di interazione, potrebbe migliorare la fruibilità. Inoltre, la sostituzione di Ngrok con una soluzione di deployment cloud permanente (es. Azure, Vercel o Firebase Hosting) permetterebbe di eliminare la dipendenza da URL temporanei durante la fase di testing. Infine, l'interfaccia grafica, già solida, potrebbe beneficiare di ulteriori ottimizzazioni UI/UX, come l'introduzione della modalità dark, transizioni animate e supporto multilingua.